

Fundamental Design and Integration of AI Agents in Hybrid Intelligence Systems

Korawit Rupanya

korawit.rupanya@aalto.fi

23.09.2026



Content is available under
CC BY-SA 4.0 unless otherwise stated

Learning objectives

- Design **the integration** of LLM services and AI agents as dependable components for HIS
- Use key **patterns** for an HIS built with AI agents with LLM services as backend and across the edge-cloud continuum
- Reason about what the integration costs
- Derive/evaluate interaction metrics among different components from observability/tracing data

1 · Introduction

2 · Design

3 · Integration aspects

4 · Observability

Running example: a Search-and-Rescue victim map generated by AI, approved by humans as a part of the service before it reaches rescuers.

Introduction

From an AI Agent

To a Dependable Component

Recall: HIS and example

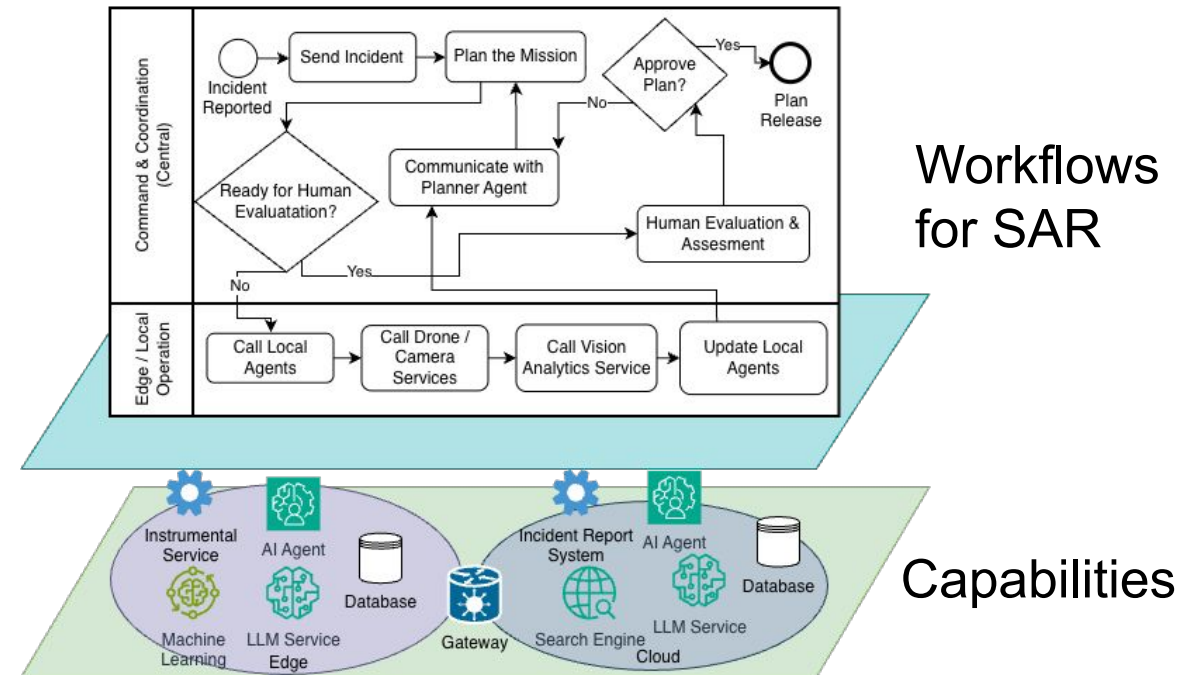
Different types of services Intelligence capabilities combination with

Different compositions for solving complex problems with regarding to

Different contexts to meet

quality, sovereignty, and compliance requirements (non-functional)

Example: SAR Operations with HIS in Edge-Cloud Continuum



Our HIS in this lecture: including LLM-based services, AI agents with LLM backends

Basic elements of HIS

- **Human as a Service:** Humans, as users or subject matter experts (SMEs), as integral components of the HIS.
- **LLM as a Service:** a LLM model provisioned under the service
 - to generate, transform, summarize, classify, and explain content based on a prompt and the context.
- **AI Agent as a Service:** a system built around a model; the agent will continue working until the task is complete.
- **Data/Compute as a Service:** Traditional AI/ML services, software modules; self-contained unit of code and responsibility that exposes a fixed interface.

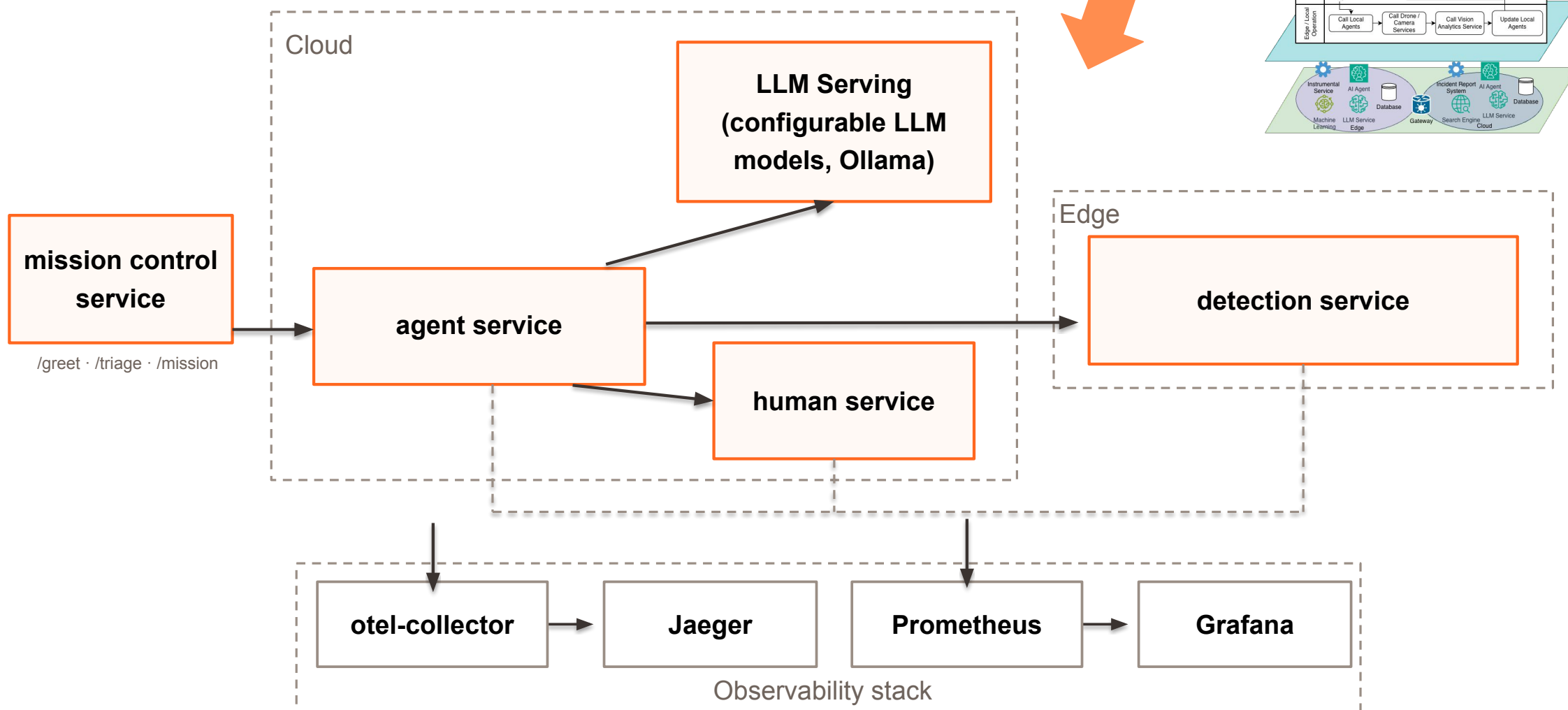
The key distinction between each element arises from its inherent characteristics, which may relate to functionality, reliability, or the degree of uncertainty it entails.

Basic interactions within HIS

Interactions among among components in HIS from a service-oriented perspective.

1. **Software Service to Software Service($S \leftrightarrow S$)** – Interactions between software, LLM-agents, and other computational services.
 - a. **AI-to-LLM interaction($ai \leftrightarrow llm$)**: between LLM-based agents and LLM services (e.g., planning agent requesting domain-specific predictions).
 - b. **AI-to-service interaction($ai \leftrightarrow service$)**: an LLM-agent to supporting software modules (e.g., retrieval services, knowledge graphs, or simulation engines).
2. **Human Service and Software Service($H \leftrightarrow S$)** – Interactions between humans and services (e.g., prompting, monitoring, or feedback).
3. **Human Service to Human service($H \leftrightarrow H$)** – Interactions between humans, mediated or facilitated by services.

The SAR HIS example: architecture



Project source: github.com/rdsea/sys4bigml/tree/master/tutorials/hisllmintegration

To build / understand HIS: what is the main concern?

- **System Behavior** - How to predict hybrid interactions behaviors?
- **Human Role & Cognitive Fit** - How to make sure the human role is optimal within HIS?
- **Evaluation Frameworks** - How to evaluate hybrid workflows? And Which metrics capture HIS performance beyond accuracy (e.g., robustness, resilience, adaptability)?
- **Patterns of Interaction** - Which interaction patterns are most effective under different conditions? And What is the effect of those patterns?

Design the Integration of the *AI Agent* for *HIS*

Using the five common patterns for HIS development

What/problem complexity

Dynamic decisions

SAR requires Human-approved critical actions

Routing and parallel tasks

SAR requires Multi-drone coordination

Multiple heterogeneous tools

SAR requires access to drone and vision services.

Probabilistic LLM responses

SAR requires consistent incident and detection information.

How and patterns

Human-in-the-Loop

decides **when to stop**, and **who signs off**

Workflow patterns; routing and parallelization

decides **who does what**, and **in what order**

Tool use through a controlled dispatcher

decides **what is allowed to actually execute**

Structured outputs; parse, validate, retry

decides **what counts as an answer**

*integration
complexity*

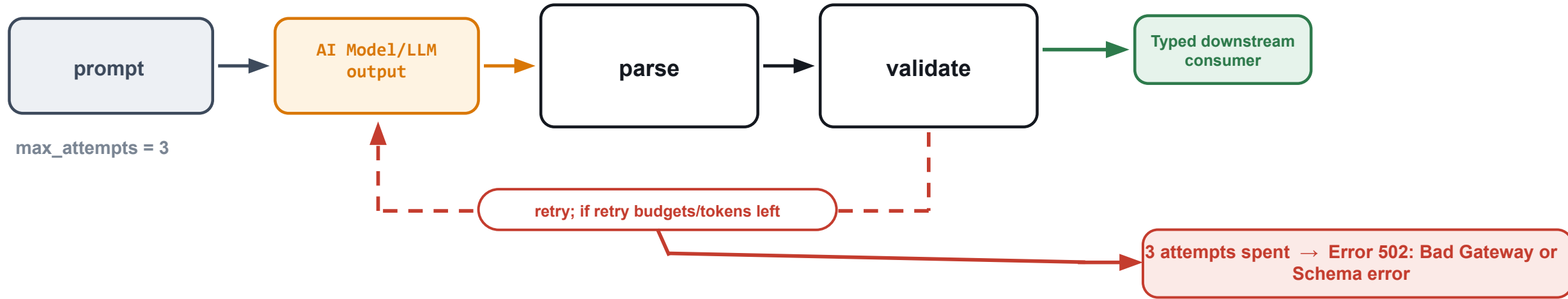


Application requirements are going up, added based on what is needed

Reading "[Agentic Design Patterns](#)", doi: 10.1007/978-3-032-01402-3 , "[Design Patterns for Multi-agent Systems: A Systematic Literature Review](#)", doi: 10.1007/978-3-642-54432-3_5

Pattern 1; Structured outputs: parse → validate → retry

also known as JSON Mode, Schema-Constrained Generation, Typed Output



Pattern Source: <https://developers.openai.com/api/docs/guides/structured-outputs?api-mode=responses>

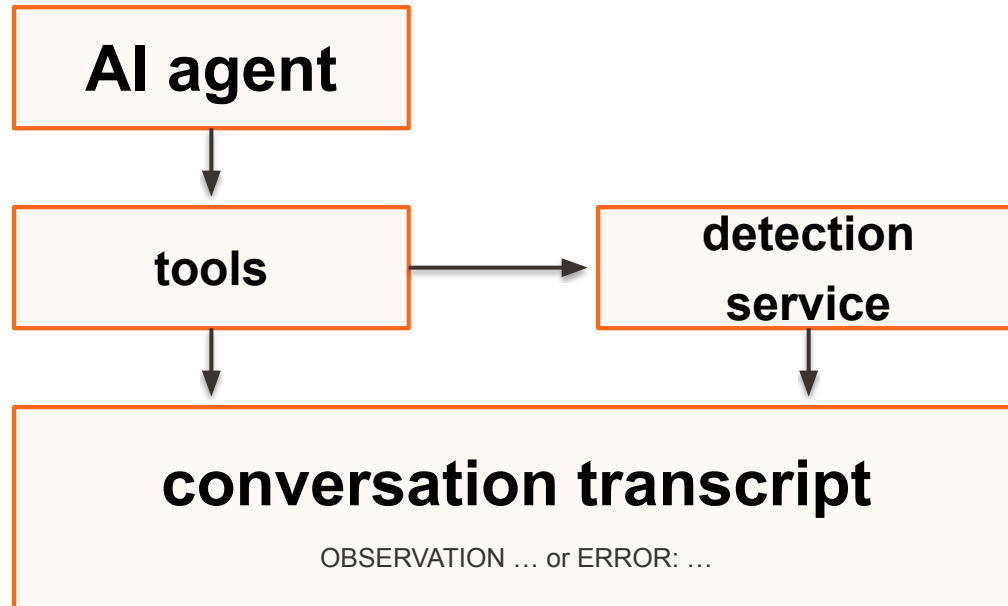
Example:

"Looking at the picture what how many people are in the picture give me the number of victims" → a retry can succeed. but "validation failed" → three identical replies.

code — <https://github.com/rdsea/sys4bigml/tree/master/tutorials/hisllmintegration> · <https://www.anthropic.com/engineering/building-effective-agents> · <https://www.agentpatterns catalog.org/patterns/structured-output>

Pattern 2; Tool use: the model requests, the dispatcher decides

also known as Function Calling, Tool Calling, Action Use



Example:

Refusing repeats

`llama3.2` fetches `weather(B1)`, gets its answer, then requests `weather(B1)` five more times.

Demanding the answer

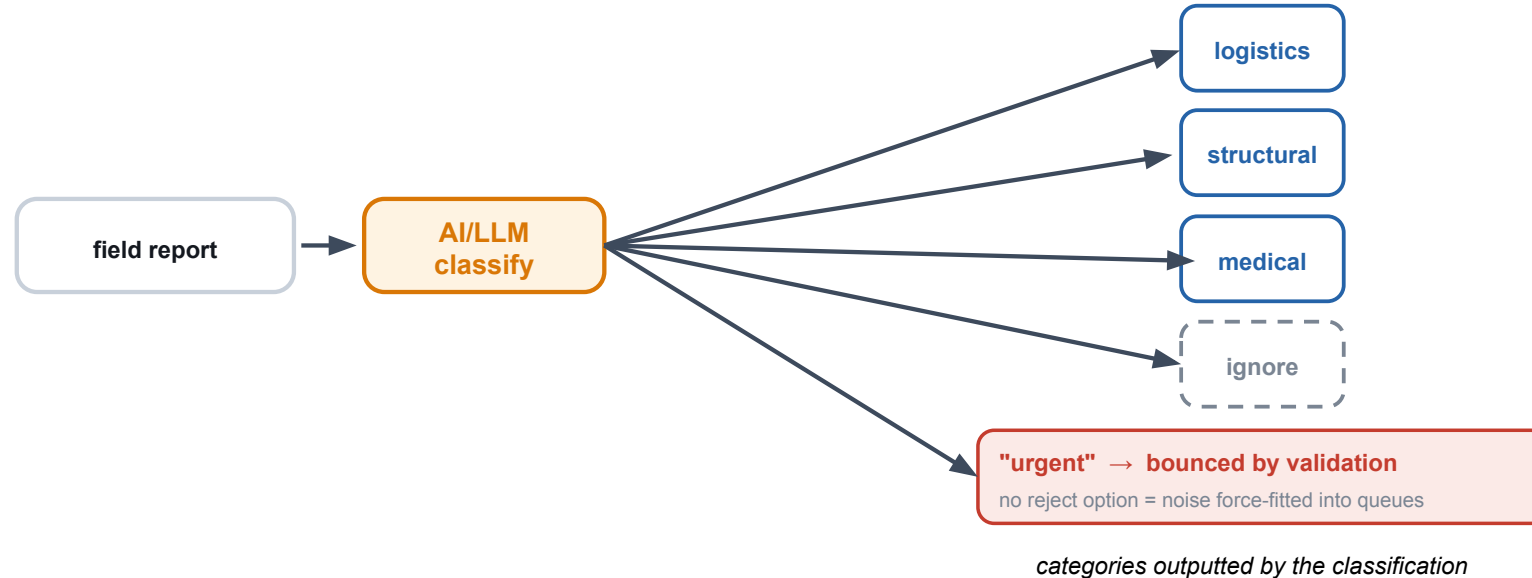
After two such stalls the tool option is withdrawn and `{"final": ...}` is required.

Pattern Source: <https://developers.openai.com/api/docs/guides/function-calling?api-mode=responses>,
<https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview>

<https://modelcontextprotocol.io/specification/2026-07-28/server/tools> · <https://genai.owasp.org/llm-top-10/> · <https://www.agentpatterns catalog.org/patterns/tool-use>

Pattern 3; Workflow patterns - Routing

also known as Mode Selector, Intent Classifier, Task Router



Pattern Source: <https://www.anthropic.com/engineering/building-effective-agents>

Example:

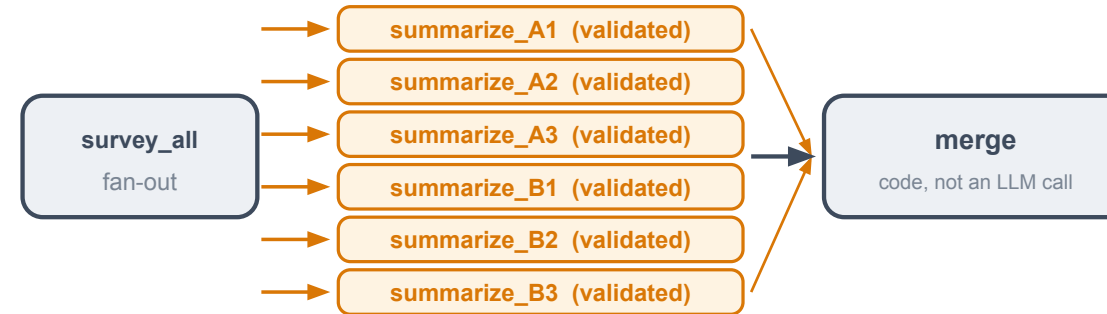
"...one is bleeding badly." / ALL CAPS / "Collapsed wall on Oak Street – two trapped." → every variant must route to medical

"What's the best restaurant near the disaster area?" / "Cat stuck in a tree." → must route to ignore

<https://docs.langchain.com/oss/python/langgraph/workflows-agents> · <https://www.anthropic.com/engineering/building-effective-agents> · <https://www.agentpatterns catalog.org/patterns/routing/>

Pattern 4; Workflow patterns - Parallelization

also known as Sectioning, Voting, Parallel Branches



Pattern Source: <https://www.anthropic.com/engineering/building-effective-agents>

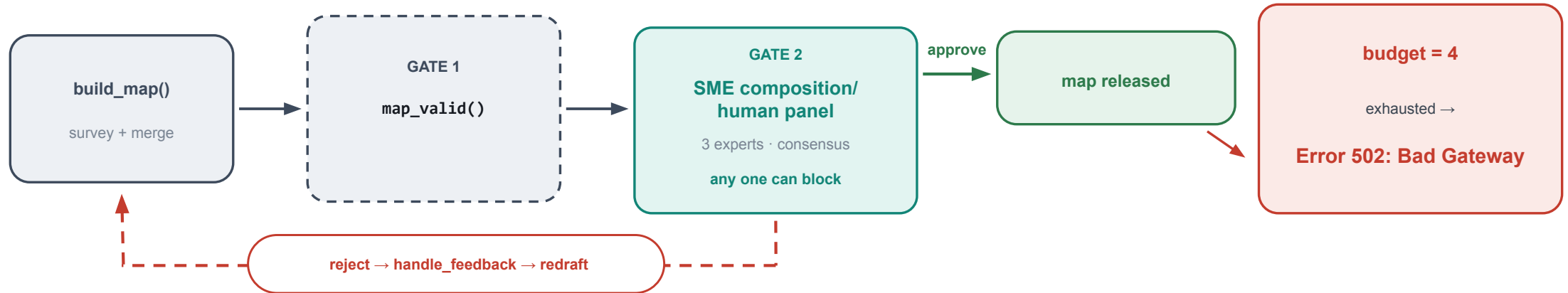
Example:

Supervisor agent asking the edge agent to summarise data from each field.

<https://langchain-ai.github.io/langgraph/tutorials/workflows/> · <https://www.anthropic.com/engineering/building-effective-agents> · <https://www.agentpatterns catalog.org/patterns/parallelization/>

Pattern 5; Human-in-the-Loop

also known as HITL, Approval Gate, Confirmation Step, Risky Action Gate, Destructive Action Confirmation, Ask Before Risky Action



Pattern Source: <https://docs.langchain.com/oss/python/langgraph/interrupts> ,

“Agent design pattern catalogue: A collection of architectural patterns for foundation model based agents”, doi: 10.1016/j.jss.2024.112278

Example:

This pattern is a pattern that identifies HIS, as a HIS must have human as part of the system. For SAR, humans can be last gate who approve or decline the plan based on their expertise together with data that that HIS give them.

<https://langchain-ai.github.io/langgraph/tutorials/workflows/> · <https://www.anthropic.com/engineering/building-effective-agents> · <https://www.agentpatterns.catalog.org/patterns/human-in-the-loop/>

Integration Aspects

Security; an untrusted component inside the trust boundary

Where and what we must pay attention to security when we do the previous patterns ...

Pattern

Security Concern

Structured outputs



Malformed / adversarial outputs

Tool use



Unauthorized tool invocation

Workflow - Routing



**Incorrect routing /
privilege escalation**

Workflow - Parallelization



**Cross-task interference /
resource abuse**

Human-in-the-Loop



Unsafe actions / gate bypass

Reading [“Agentic Design Patterns”](#), doi: 10.1007/978-3-032-01402-3 , [“Design Patterns for Multi-agent Systems: A Systematic Literature Review”](#), doi: 10.1007/978-3-642-54432-3_5

Security: example with SAR

The question is never “can it be injected?”; it can but “what is the worst it can cause, and where does that stop?”

CHANNEL	PROMPT IT ENTERS	WORST CASE	WHERE IT STOPS
field reports	TRIAGE_PROMPT	a wrong label for the report	4 categories for the field report; code decides what a label triggers
edge detections	SUMMARY_PROMPT	- attacker-controlled prompt text - payload injection	validate_summary recomputes count and confidence; then human consensus
rejection reasons	handle_feedback → tools	The gate sighting is bad.	- non-matching text is ignored - strict regex sector in different geographically

In all three rows the defense mechanisms are the same two things: a limited vocabulary, and a decision made in code. Neither is about the AI Agent.

<https://genai.owasp.org/llmrisk/llm01-prompt-injection/> · <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>

Performance; every defence costs

Where and what we must pay attention to security when we do the previous patterns ...

Pattern

Performance Concern

Structured outputs



Validation and retry overhead

Tool use



Dispatcher latency

Workflow - Routing



Routing decision overhead

Workflow - Parallelization



Resource contention

Human-in-the-Loop

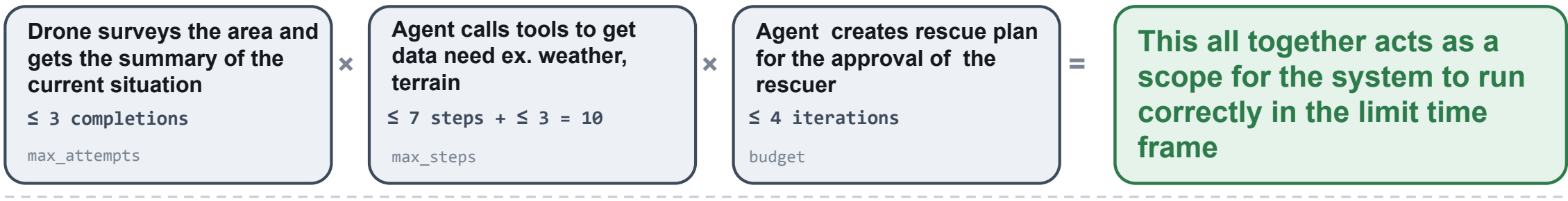


Human decision latency

Reading "[Agentic Design Patterns](#)", doi: 10.1007/978-3-032-01402-3 , "[Design Patterns for Multi-agent Systems: A Systematic Literature Review](#)", doi: 10.1007/978-3-642-54432-3_5

Performance: example with SAR

Worst-case cost is computable by system design before the execution.



THE INTERACTION MIX INVERTS WITH A MODEL



`agent_llm_retries_total / agent_llm_requests_total`
the price of model unreliability, in one number; the leading indicator to watch after every prompt edit and every model swap.

Elements have a timeout; tools 10 (s), human review 30 (s), Ollama 120 (s). An underperforming dependency becomes a failure as design inside a finished trace, never a silently stuck mission.

Let's start the hands-on

OBSERVABILITY

MAKING THE DECISIONS MONITORABLE

Every model call, tool dispatch, retry and human decision is a child span inside a trace span.

Now look at what a child span actually is.

The three child span that make AI Agent behaviour inspectable (S2S)

Three child span make AI Agent behaviour inspectable. So in order to cover the observability of HIS, we shall make use of them with mapping them to the interaction types.

llm.complete

only the code at this point knows the task

A2L

```
llm.task          = summarize_A1
llm.is_retry      = true
llm.output.preview = "{\\"victims\\"..."
```

tool.get_detections

the hop itself is the evidence

A2S

```
tool.args = {"sector": "A1"}
http.url, http.status_code
← free, from RequestsInstrumentor
```

human.review

and inside: vote_by_* spans = H2S

H2S

```
decision = reject
reason   = "recheck sector A1"
duration = seconds of attention
```

Two tag vocabularies, only one portable: `span.kind` · `http.url` = OpenTelemetry semantic conventions
`llm.task` · `llm.is_retry` = ours, invented here. In production prefer `gen_ai.*`

<https://opentelemetry.io/docs/specs/semconv/gen-ai/> · <https://langfuse.com/docs>

Conclusion

- **Everything in a HIS is enforced outside the LLM**; the LLM suggests, the code decides what to be used.
- The integration patterns: structured outputs, a controlled tool dispatcher, workflow patterns, and Human-in-the-Loop.
- Enhancing AI Agent is to put inside a trust boundary: least agency, output as data, bounded misbehaviour, auditability.
- Observability enables us to manage risk that might happen based on specific patterns and system scenarios.

Challenges

- Where in the system would you put the human and what would you give up to move it earlier or later?
- Will the system survive replacing the AI Agent with a human, and which become meaningless?
- Remove one mechanism to cut cost. Which one, and what must you now monitor that you did not before?
- How does all of this link back to resilience, reliability, robustness and elasticity in a multi-continuum system?

Study log

- If you want to add more patterns, what concerns or requirements would they address? Would you need additional monitoring? If so, what would you monitor, and what would change in terms of security?
- Write a short reflection and send it to MyCourses.
- Choose one and write short thought and sent it to MyCourses.

Reading;

- A Practical Guide for Designing, Developing, and Deploying Production-Grade Agentic AI Workflows
- From Static Templates to Dynamic Runtime Graphs: A Survey of Workflow Optimization for AI Agents
- Observability and Fault Injection for LLM-Based Multi-Agent Systems in Software Engineering
- AI Observability for Large Language Model Systems: A Multi-Layer Analysis of Monitoring Approaches from Confidence Calibration to Infrastructure Tracing

A!

Kiitos
aalto.fi